

HDI Predictor Documentation

HDI Predictor: Human Development Index Classification System

Date	08 July 2025
Team ID	SB-GENAI-HDI-2025
Project Name	HDI Predictor - Human Development Index Classification System
Prepared By	Kankipati Sai Venkat

Project Description

The Human Development Index (HDI) is a statistical composite index of life expectancy, education, and per-capita income indicators, used to rank countries into four tiers — Very High, High, Medium, and Low human development. HDI Predictor is a machine-learning-powered web application that takes a country’s (or hypothetical scenario’s) core indicators — life expectancy, mean years of schooling, expected years of schooling, and GNI per capita (PPP) — and instantly returns its predicted development tier along with a numeric HDI estimate.

The system is built around a **RandomForestClassifier** trained on historical country-level HDI data, achieving **97.6% accuracy** on the held-out test set. The model is served through a **Flask** web application with a simple input form and results panel, and the entire solution is packaged with **Docker** so it can be deployed and evaluated consistently on any machine.

Scenarios

Scenario 1 — Predicting Very High Human Development: A user selects a country with high life expectancy, strong mean and expected years of schooling, and a high GNI per capita. The model predicts a **Very High HDI** score, classifying the country among the most developed nations, helping the user see how strong performance across all three dimensions compounds into overall development.

Scenario 2 — Identifying Development Gaps in Emerging Economies: A policymaker inputs mid-range values representing a developing nation — moderate life expectancy, average educational attainment, and moderate income. The model returns a **Medium HDI** score and, via the feature-importance summary, highlights which dimension (health, education, or income) is holding the country back — pointing toward where policy investment would have the most impact.

Scenario 3 — Assessing Countries Requiring Development Intervention: A researcher evaluates a country with relatively low life expectancy, limited educational opportunities, and low GNI per capita. The model predicts a **Low HDI** score, flagging the country’s developmental challenges and supporting governments and development organizations in prioritizing investments and interventions.

Technical Architecture

HDI Predictor follows a modular three-tier architecture. The **presentation layer** is a lightweight HTML/CSS form served through Flask’s Jinja2 templates. The **application layer** (Flask backend) validates incoming indicator values, applies the same preprocessing/scaling used during training, and calls the trained RandomForest model to generate a prediction. The **data layer** holds the historical training dataset and the serialized model artifact (`model.pkl`), which is loaded once at startup for fast repeated inference. The entire stack is containerized with Docker for reproducible deployment.

(No relational database is used in this project — indicator data is tabular and file-based, so no ER diagram is required.)

Pre-requisites

- **Python Programming Proficiency:** [Python Documentation](#)
- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Machine Learning with scikit-learn:** [scikit-learn Documentation](#)
- **Data Handling with Pandas/NumPy:** [Pandas Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Containerization with Docker:** [Docker Documentation](#)

Project Workflow

Milestone 1: Data Collection & Exploratory Analysis - Activity 1.1: Collect a country-level dataset containing life expectancy, mean years of schooling, expected years of schooling, GNI per capita, and the labeled HDI tier/score. - Activity 1.2: Clean the dataset — handle missing values, standardize units, and remove duplicate country-year records. - Activity 1.3: Perform exploratory data analysis (EDA) to understand indicator distributions and their correlation with HDI tier, using histograms, boxplots, and a correlation heatmap.

Milestone 2: Model Selection & Training - Activity 2.1: Split data into training and test sets with stratification to preserve tier proportions. - Activity 2.2: Train and compare baseline models (Logistic Regression, Decision Tree) against a RandomForestClassifier. - Activity 2.3: Tune RandomForest hyperparameters (`n_estimators`, `max_depth`, `min_samples_split`) via cross-validation. - Activity 2.4: Select the final RandomForestClassifier configuration, which reached **97.6% test accuracy**. - Activity 2.5: Serialize the trained model and any preprocessing objects (scaler) with `joblib` for reuse.

Milestone 3: Flask Application Development - Activity 3.1: Build `app.py` with a `/` route for the input form and a `/predict` route that accepts POSTed indicator values. - Activity 3.2: Implement input validation to reject missing or out-of-range values with a friendly error message. - Activity 3.3: Load the serialized model once at application startup and reuse it across requests for low latency. - Activity 3.4: Implement the feature-importance summary so the results page can explain which indicator most influenced the prediction.

Milestone 4: Frontend Development - Activity 4.1: Design a simple, clean HTML/CSS form for entering the four indicators, with inline hints on expected units and ranges. - Activity 4.2: Build the results template to display the predicted tier, HDI score estimate, and feature-importance breakdown.

Milestone 5: Deployment - Activity 5.1: Write a `Dockerfile` and `requirements.txt` capturing all dependencies (Flask, scikit-learn, pandas, numpy, joblib). - Activity 5.2: Build and run the Docker image locally, verifying the app is reachable at `http://127.0.0.1:5000`. - Activity 5.3: Test the containerized app end-to-end with all three scenarios (Very High, Medium, Low) to confirm consistent behavior.

Milestone 6: Testing & Documentation - Activity 6.1: Run performance tests on the `/predict` endpoint to confirm sub-second response times. - Activity 6.2: Write the README with setup, run, and usage instructions. - Activity 6.3: Compile the full documentation package covering ideation, requirements, design, planning, development, testing, and demonstration phases.

Home / Input Page

Description: The single-page interface serves as both the landing page and the prediction tool. It presents a clean form with four numeric fields — Life Expectancy (years), Mean Years of Schooling, Expected Years of Schooling, and GNI per Capita (PPP, USD) — along with short helper text describing typical ranges, and a **Predict** button.

Results Page

Description: After submission, the results panel displays the predicted **HDI Tier** (Very High / High / Medium / Low) in a color-coded badge, the estimated **HDI score** (0–1 scale), and a short feature-importance summary explaining which input contributed most to the classification — helping users understand not just *what* the prediction is, but *why*.

Conclusion

HDI Predictor demonstrates how a modest, well-understood dataset and a single well-tuned classical ML model (RandomForestClassifier, 97.6% accuracy) can be turned into a genuinely useful decision-support tool. By pairing the model with a lightweight Flask interface and Docker packaging, the project delivers instant, explainable, and reproducible human-development classification for researchers, policymakers, and students. Future enhancements could include multi-country comparison views, historical trend visualization, a regression head for finer-grained score prediction, and public cloud hosting for wider accessibility.